

Design Patterns Créationnels en PHP

Objectif des patterns créationnels

Ils permettent de **contrôler la création d'objets** :

- Centraliser la logique d'instanciation
 - Éviter les duplications de code
 - Rendre la création d'objets flexible et paramétrable
 - Préparer à des évolutions (remplacement d'une classe par une autre, injection de dépendances, etc.)
-

Sommaire :

1. [Singleton](#)
 2. [Factory Method](#)
 3. [Abstract Factory](#)
 4. [Builder](#)
 5. [Prototype](#)
-

1. Singleton

 But :

Garantir qu'une **classe n'a qu'une seule instance** et fournir un point d'accès global à celle-ci.

 Exemple en PHP :

```
class Database {
    private static $instance = null;
    private function __construct() {
        echo "Connexion créée\n";
    }

    public static function getInstance(): Database {
        if (self::$instance === null) {
            self::$instance = new Database();
        }
        return self::$instance;
    }
}

// Utilisation
$db1 = Database::getInstance();
$db2 = Database::getInstance();

var_dump($db1 === $db2); // true
```

 Avantages :

- Contrôle centralisé
- Évite les doublons de connexion, de logs, etc.

 Inconvénients :

- Couplage fort
- Difficile à tester en unit tests (pas de reset facile)

2. Factory Method

 But :

Définir une **interface pour créer un objet**, mais laisser les sous-classes **décider quelle classe instancier**.

 Exemple en PHP :

```
abstract class Transport {
    abstract public function livrer(): string;
}

class Camion extends Transport {
    public function livrer(): string {
        return "Livraison par camion";
    }
}

class Bateau extends Transport {
    public function livrer(): string {
        return "Livraison par bateau";
    }
}

class Logistique {
    public static function creerTransport(string $type): Transport {
        return match($type) {
            'camion' => new Camion(),
            'bateau' => new Bateau(),
            default => throw new Exception("Type inconnu")
        };
    }
}

// Utilisation
$t = Logistique::creerTransport('camion');
echo $t->livrer(); // Livraison par camion
```

✓ Avantages :

- Encapsulation de la création
- Facile à étendre

3. 🏭 Abstract Factory

📌 But :

Fournir une **interface pour créer des familles d'objets liés entre eux**, sans spécifier leurs classes concrètes.

📦 Exemple en PHP :

```
// Interfaces
interface Bouton { public function render(): string; }
interface Checkbox { public function render(): string; }

// Implémentations Windows
class BoutonWindows implements Bouton {
    public function render(): string { return "Bouton Windows"; }
}
class CheckboxWindows implements Checkbox {
    public function render(): string { return "Checkbox Windows"; }
}

// Implémentations Mac
class BoutonMac implements Bouton {
    public function render(): string { return "Bouton Mac"; }
}
class CheckboxMac implements Checkbox {
    public function render(): string { return "Checkbox Mac"; }
}

// Fabrique
interface GUIFactory {
    public function createButton(): Bouton;
    public function createCheckbox(): Checkbox;
}

class MacFactory implements GUIFactory {
    public function createButton(): Bouton { return new BoutonMac(); }
    public function createCheckbox(): Checkbox { return new CheckboxMac(); }
}

class WindowsFactory implements GUIFactory {
    public function createButton(): Bouton { return new BoutonWindows(); }
    public function createCheckbox(): Checkbox { return new
CheckboxWindows(); }
}
```

```
// Utilisation
function renderUI(GUIFactory $factory) {
    $btn = $factory->createButton();
    $chk = $factory->createCheckbox();
    echo $btn->render() . "\n";
    echo $chk->render() . "\n";
}

$sui = new MacFactory();
renderUI($sui);
```

✅ Avantages :

- Compatible avec l'architecture MVC
- Centralisation par contexte (ex. : thème, système, plateforme)

4. 🧱 Builder

📌 But :

Séparer la **construction complexe** d'un objet de sa **représentation finale**.

🧱 Exemple en PHP :

```
class Pizza {
    public $pate;
    public $garniture;

    public function afficher() {
        echo "Pizza avec $this->pate et $this->garniture\n";
    }
}

class PizzaBuilder {
    private $pizza;

    public function __construct() {
        $this->pizza = new Pizza();
    }

    public function ajouterPate(string $pate): self {
        $this->pizza->pate = $pate;
        return $this;
    }

    public function ajouterGarniture(string $garniture): self {
        $this->pizza->garniture = $garniture;
        return $this;
    }
}
```

```

    public function construire(): Pizza {
        return $this->pizza;
    }
}

// Utilisation fluide
$p = (new PizzaBuilder())
    ->ajouterPate("fine")
    ->ajouterGarniture("fromage")
    ->construire();

$p->afficher(); // Pizza avec fine et fromage

```

✅ Avantages :

- Chaîne fluide (**method chaining**)
- Construction progressive, utile pour les objets complexes (PDF, requêtes SQL...)

5. Prototype

But :

Cloner un objet existant au lieu de le créer depuis zéro.

Exemple en PHP :

```

class Formulaire implements Cloneable {
    public $titre;
    public $champs = [];

    public function __clone() {
        // Profonde copie si nécessaire
        $this->champs = array_map(fn($c) => clone $c, $this->champs);
    }
}

class Champ {
    public $nom;
    public function __construct($nom) {
        $this->nom = $nom;
    }
}

// Utilisation
$form = new Formulaire();
$form->titre = "Inscription";
$form->champs[] = new Champ("email");

$form2 = clone $form;

```

```
$form2->titre = "Connexion";  
  
var_dump($form, $form2);
```

✅ Avantages :

- Clonage rapide d'objets configurés
- Gain de performances quand instancier est coûteux